

Lógica e Inteligencia Artificial -

Práctica 8 - Agentes Inteligentes y

Prolog

Temario

- Agentes inteligentes basados en conocimiento. Programación lógica. Prolog.

Bibliografía

- Inteligencia artificial. Un enfoque moderno. Russell y Norvig. Cap 7.

Fecha de entrega

17 de noviembre de 2025.

Integrantes

- Lautaro Josin Saller
- Juan Cruz Cassera Botta
- Juan Cruz Mazullo
- Mariano Gonzalez

Ejercicios

1. Para el mundo del Wumpus descrito en Russell y Norvig, vimos algunas reglas que lo caracterizan, por ejemplo “En una casilla se siente una brisa si y sólo si hay un pozo en una casilla vecina”. Escriba en lenguaje natural y en lenguaje simbólico al menos dos reglas adicionales para caracterizar el mundo del Wumpus.

Regla 1:

En lenguaje natural: El agente verá un resplandor si y sólo si hay oro en una casilla vecina.

En lenguaje simbólico:

Esta regla se ha de especificar para cada casilla, pero a modo de ejemplo se usarán únicamente las casillas [1,1] y [2,1]

Vocabulario de símbolos proposicionales:

- $O_{i,j}$ será verdadero si hay oro en la casilla [i,j]
- $R_{i,j}$ será verdadero si hay un resplandor en la casilla [i,j]

R1: $\neg O_{1,1}$

R2: $R_{1,1} \leftrightarrow (O_{1,2} \vee O_{2,1})$

R3: $R_{2,1} \leftrightarrow (O_{2,2} \vee O_{3,1})$

Regla 2:

En lenguaje natural: Si el wumpus es aniquilado emite un desconsolado grito que se puede oír en toda la cueva.

En lenguaje simbólico:

Vocabulario de símbolos proposicionales:

- Sea w la constante para representar al Wumpus
- Sea $A(x)$: “x es aniquilado”
- Sea $S(x)$: “x emite un desconsolador grito”
- $G_{i,j}$ será verdadero si se escucha el grito del wumpus en la casilla [i,j]

R4: $A(w) \rightarrow S(w) \wedge (\forall i)(\forall j)G_{i,j}$

2. **Diseñar e implementar en Prolog un agente que asista en las búsquedas laborales. Especifique el REAS del agente. Se ha realizado la siguiente encuesta para determinar las habilidades y preferencias de los postulantes:**

Encuesta para empresa de software:

Nombre:

nombre

Tengo los siguientes

Skills:

- ☒ Gestión de Recursos Humanos
- ☒ Buena Comunicación
- ☐ Empatía con Clientes
- ☒ UML
- ☐ Arquitecturas
- ☒ java
- ☐ javascript
- ☐ PHP
- ☐ C/C++
- ☐ PYTHON

Me gusta trabajar
como:

- ☒ Lider
- ☐ Analista
- ☒ Diseñador
- ☐ Programador

Me puedo trasladar
hasta:

- ☐ Capital
- ☒ Córdoba
- ☐ Mendoza
- ☒ Al exterior

Enviar

En base a estos datos y considerando que un postulante podrá ocupar un rol si lo ha expresado como preferencia y si posee alguna habilidad relacionada con dicho rol, se desea consultar:

- Las personas disponibles para ocupar un cierto rol en una determinada localidad.
- Las personas disponibles para un cierto rol.
- Listar los roles que puede ocupar una persona dada.
- Si una persona en particular es apta para un cierto rol.

REAS del agente que asiste en las búsquedas laborales:

- **Rendimiento:** Cantidad de trabajos encontrados, minimizar tiempo de búsqueda, Tasa de éxito en la postulación
- **Entorno:** Sitio web del ministerio de trabajo, Linkedin, Usuarios
- **Actuadores:** Pantalla para mostrar resultados, servidor para realizar consultas HTTPS por internet
- **Sensores:** Formulario para que completen los usuarios, teclado, mouse

HECHOS:

```
% Relación entre skills y roles
es_necesario(comunicacion, lider).
es_necesario(empatia, lider).
es_necesario(gestion, lider).
es_necesario(arquitecturas, lider).

es_necesario(comunicacion, analista).
es_necesario(uml, analista).

es_necesario(uml, diseñador).
es_necesario(arquitecturas, diseñador).

es_necesario(java, programador).
es_necesario(javascript, programador).
es_necesario/php, programador).
es_necesario(c, programador).
es_necesario(python, programador).

% Persona pedro y tomás

posee_skill(pedro, comunicacion).
posee_skill(pedro, uml).
posee_skill(pedro, java).
posee_skill(tomas, comunicacion).

prefiere_rol(pedro, lider).
prefiere_rol(pedro, diseñador).
prefiere_rol(tomas, analista).
```

```
se_puede_trasladar(pedro, cordoba).
se_puede_trasladar(pedro, exterior).
se_puede_trasladar(tomas, salta).
```

REGLAS:

```
empleado_disponible(Persona, Rol, Ubicacion):-
    se_puede_trasladar(Persona, Ubicacion),
    es_apto(Persona, Rol).
```

```
es_apto(Persona, Rol) :-
    prefiere_rol(Persona, Rol),
    posee_skill(Persona, Skill), es_necesario(Skill, Rol).
```

Explicado en lenguaje natural:

- Un empleado es apto para un cierto rol si lo prefiere y si posee la skill necesaria para el mismo.
- Un empleado está disponible para un cierto rol en una determinada ubicación si es apto y si se puede trasladar a la misma.

[Link al programa en Prolog](#)

Resultados de algunas ejecuciones:

Las personas disponibles para ocupar un cierto rol en una determinada localidad: A partir de esta consulta, Prolog buscará a todas las personas aptas para un cierto rol en una determinada localidad. Como no se instancio (asignó valor) ninguna variable en la consulta, se listan todos los resultados posibles, pero si se asignaran valores a las variables se podría conocer si una determinada persona es apta para un cierto rol en una ubicación específica.

```
empleado_disponible(Persona,Rol,Ubicacion)

Persona = pedro,
Rol = lider,
Ubicacion = cordoba
Persona = pedro,
Rol = diseñador,
Ubicacion = cordoba
Persona = pedro,
Rol = lider,
Ubicacion = exterior
Persona = pedro,
Rol = diseñador,
Ubicacion = exterior
Persona = tomas,
Rol = analista,
Ubicacion = salta
Persona = tomas,
Rol = analista,
Ubicacion = salta

?- empleado_disponible(Persona,Rol,Ubicacion)
```

Las personas disponibles para un cierto rol: En este caso, el objetivo fue obtener las personas disponibles para ser diseñadoras. Para eso se usó la constante diseñador y no se instanció la variable Persona.

<pre>es_apto(Persona,diseñador) Persona = pedro false ?- es_apto(Persona,diseñador)</pre>	<pre>es_apto(Persona,analista) Persona = tomas ?- es_apto(Persona,analista)</pre>
---	---

Listar los roles que puede ocupar una persona dada: En esta consulta, el objetivo fue listar los roles que pedro y tomas pueden ocupar ya que la variable Rol no fue instanciada. Prolog utilizó backtracking para encontrar todos los valores posibles que puede tomar la variable rol en cada caso.

 <code>es_apto(pedro,Rol)</code>	 <code>es_apto(tomas,Rol)</code>
<code>Rol = lider</code>	<code>Rol = analista</code>
<code>Rol = diseñador</code>	
<code>false</code>	
<code>?- es_apto(pedro,Rol)</code>	<code>?- es_apto(tomas,Rol)</code>

Si una persona en particular es apta para un cierto rol: En este último caso, el objetivo era saber si la persona Tomás era apto para ser líder y si Tomás era apto para ser analista.

 <code>es_apto(tomas,lider)</code>	 <code>es_apto(tomas,analista)</code>
<code>false</code>	<code>true</code>
<code>?- es_apto(tomas,lider)</code>	<code>?- es_apto(tomas,analista)</code>

3. Diseñar e implementar en Prolog un agente que de información sobre el árbol genealógico de las personas

- Escribir en Prolog los hechos que definen algunas relaciones familiares directas (ej, juan es padre de pedro, ana es madre de juana, etc).
- Definir la relación progenitor, utilizando las relaciones de padre y madre.
- Definir recursivamente la relación antepasado.
- Definir nuevas relaciones (como hermano, hermana, abuelo, abuela) añadiendo los predicados (por ejemplo mujer, hombre) y reglas necesarios.

[Link a la solución en Prolog \(swi-prolog.org\).](http://swi-prolog.org)

Explicación de la solución:

La solución se divide entre los hechos/relaciones y las reglas.

Hechos/relaciones:

- `hombre(x)`
 - Define que x es un hombre.
- `mujer(x)`
 - Define que x es una mujer.
- `padre(x, y)`
 - Define que x es el padre de y.
- `madre(x, y)`
 - Define que x es la madre de y.

Reglas:

- `progenitor(A, B) :- padre(A, B); madre(A, B).`
 - A es progenitor de B si A es el padre de B o A es la madre de B.
- `antepasado(A, B) :- progenitor(A, B).`
 - (Caso base de la regla recursiva) A es antepasado de B si A es progenitor de B.
- `antepasado(A, B) :- progenitor(A, C), antepasado(C, B).`
 - (Caso recursivo de la regla recursiva) A es antepasado de B si A es progenitor de C y además C es antepasado de B.
- `abuelo(A, B) :- padre(A, C), padre(C, B).`
 - A es abuelo de B si A es padre de C y C es padre de B (abuelo paterno).
- `abuelo(A, B) :- padre(A, C), madre(C, B).`
 - Además, A es abuelo de B si A es padre de C y C es madre de B (abuelo materno).
- `abuela(A, B) :- madre(A, C), madre(C, B).`
 - A es abuela de B si A es madre de C y C es madre de B (abuela materna).
- `abuela(A, B) :- madre(A, C), padre(C, B).`
 - Además, A es abuela de B si A es madre de C y C es padre de B (abuela paterna).
- `hermano(A, B) :- hombre(A), progenitor(X, A), progenitor(X, B), A \== B.`
 - A es hermano de B si A es un hombre, A y B comparten un mismo progenitor, y además A no es la misma persona que B.
- `hermana(A, B) :- mujer(A), progenitor(X, A), progenitor(X, B), A \== B.`
 - A es hermana de B si A es una mujer, A y B comparten un mismo progenitor, y además A no es la misma persona que B.

Resultados de algunas ejecuciones:

 hermano(pedro, camila).	 antepasado(martin, pedro).
true	true
 hermana(camila, pedro).	 antepasado(martin, juan).
true	true
 hermana(camila, camila).	 progenitor(ana, camila).
false	true
 hermano(pedro, pedro).	 progenitor(juan, camila).
false	true
 abuelo(martin, juan).	 abuela(carla, pedro).
true	true
 abuelo(martin, pedro).	
false	

4. Implementar en Prolog el ejercicio de la práctica 4:

Si alguien hace algo bueno, ese alguien es bueno. Del mismo modo, si alguien hace algo malo, es malo. Sebastián ayuda a su madre y también miente algunas veces. Mentir es malo y ayudar es bueno.

Determinar si con el conocimiento disponible es posible deducir que Sebastián es bueno. ¿Y es posible deducir que es malo?

De la misma forma que los ejercicios anteriores vamos a definir hechos y reglas para poder realizar la deducción lógica.

En función de las bases de verdad que nos explica el enunciado definimos lo siguiente:

Definición de los predicados:

- hace(x, y) : Define que x hace la acción y
- es_bueno(x): Define que la acción x es buena
- es_malo(x): Define que la acción x es mala

Hechos

- `hace(sebastian, ayudar)`
- `hace(sebastian, mentir)`
- `es_bueno(ayudar)`
- `es_malo(mentir)`

Reglas

- `bueno(Persona) :-`
 `hace(Persona, Accion),`
 `es_bueno(Accion)`
Una persona es buena si hace algo bueno
- `malo(Persona) :-`
 `hace(Persona, Accion),`
 `es_malo(Accion).`
Una persona es mala si hace algo malo

Cuando ejecutamos el código obtenemos la siguiente salida:

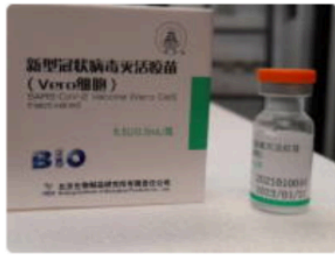
```
 bueno(sebastian)  
  
true  
  
  
  
 malo(sebastian)  
  
true
```

Es decir que existe al menos un camino lógico para poder determinar que Sebastián es bueno. Además también existe un camino lógico para determinar que es malo.

5. Implementar en Prolog un agente que asista en la distribución de la vacuna contra el COVID-19. Su base de conocimiento debe ser construida a partir de la especificación de cada una de las vacunas que se administran, disponible en el sitio [¿Cuáles vacunas aplicamos en el país? | Argentina.gob.ar](https://www.argentina.gob.ar/salud/vacunacion)

Por ejemplo, en el caso de la siguiente vacuna:

Sinopharm



Nombre: SARS COV-2 (células vero) inactivada

Desarrollador: Beijing Institute of Biological Products - República Popular China

Autorizado edad (edad): ≥ 3 años

Plataforma: virus inactivados

Contraindicaciones: hipersensibilidad a cualquier componente; antecedente de reacciones alérgicas graves (con compromiso respiratorio que haya requerido asistencia médica); exacerbación de enfermedades crónicas, que impliquen compromiso del estado general.

Contraindicación para la segunda dosis: reacción anafiláctica con la primera dosis.

Se podrían definir los siguientes hechos:

`edad_minima(sinopharm, 3).`

`contraindicacion(sinopharm, hipersensibilidad).`

`contraindicacion(sinopharm, enfermedades_cronicas).`

`contraindicacion(sinopharm, trombocitopenia).`

`contraindicacion(sinopharm, trastornos_coagulación).`

`contraindicacion(sinopharm, epilepsia).`

Crear una base de hechos con las edades y comorbilidades de un grupo de personas. Escribir las reglas necesarias para que el agente pueda determinar cuál vacuna puede aplicarse a cada persona.

<https://swish.swi-prolog.org/p/MltNKNUt.pl>

- Los **hechos** de *edad_minima* representan la edad mínima (en años) que debe tener una persona para ser apto para una vacuna.
- Los **hechos** de *contraindicación* representan las contraindicaciones que presenta cada vacuna.
- Los **hechos** de *Persona* representan las diferentes personas, con su nombre, edad y un listado de contraindicaciones

- La **regla** de *hay_contraindicacion* determina si al menos una de las condiciones de la persona es una contraindicación para una vacuna.
- La **regla** *vacuna_apta* determina si una vacuna es apta para una persona.

Resultados de algunas ejecuciones:

 <i>vacuna_apta</i> (juan, sinopharm)	  
true	1
 <i>vacuna_apta</i> (juan, comirnaty)	  
false	
 <i>vacuna_apta</i> (juan, moderna)	  
true	1
 <i>vacuna_apta</i> (pedro, sinopharm)	  
true	1
 <i>vacuna_apta</i> (pedro, moderna)	  
false	
 <i>vacuna_apta</i> (pedro, comirnaty)	  
true	1
 <i>vacuna_apta</i> (juana, sinopharm)	  
false	
 <i>vacuna_apta</i> (juana, moderna)	  
true	1
 <i>vacuna_apta</i> (juana, comirnaty)	  
false	